Project Plan — Electricity Tariff and Billing Dispute Triage Service

UE24AM342AA5 ML System Design and AgentOps · Mini Project 11 · Jackfruit (Level 3) · Evaluation 1

Team: Dhanush S (PES1UG24AM360) · Vidit Soni (PES1UG24AM318)

1. Objective and scope

We will build a multi-agent service that accepts a consumer's electricity billing dispute, validates it at the ingestion boundary, works out the correct tariff category from the Karnataka tariff order and supply code, and routes the case to **bill-stands-with-reason**, **revision-recommended** or **site-inspection-required** with clause-level citations. When the category is genuinely mixed, provisions conflict, or the connection type is outside our corpus, it **escalates to a grievance officer** instead of answering. The system never revises or recomputes a bill. Success means the section 4.1 tailoring-shop case is answered with both governing provisions cited (or escalated with a stated reason), and no unverified answer is ever returned.

2. Corpus (state: Karnataka, KERC)

Document	What we need from it	Handling
KERC retail supply tariff order currently in force for BESCOM	Category definitions (LT-2(a) domestic, LT-3 commercial, LT-5 LT industrial) and the notes under each category	Text-layer PDF, parsed with PyMuPDF
KERC Conditions of Supply of Electricity of Distribution Licensees in Karnataka (with amendments)	Change of use / partly non-domestic premises, reclassification, inspection and back-billing provisions	Text layer checked page by page; Tesseract OCR only for scanned pages

Each file goes into a manifest (source URL, version/date, SHA-256, download date), which feeds the data-freshness metric. **Section 4.1 case:** at Evaluation 1 we will show (a) the LT-2(a) domestic definition, (b) the provision on premises used partly for a non-domestic purpose, which sits away from the definition, and (c) the category a tailoring activity falls under. Test values: domestic LT connection, 2 kW sanctioned load, billed under LT-2(a), one room used as a tailoring shop with one 0.25 kW machine motor. The outcome confirmed at Evaluation 1 becomes our reference answer.

3. Architecture (Docker Compose, single `docker compose up`)

Service	Responsibility	Why it is its own boundary
api (FastAPI)	Data contract (Pydantic), validation, quarantine, POST /disputes → 202 + run_id, GET /runs/{id}, POST /feedback, /health, /metrics, per-client rate limit, serves the web client	Must stay responsive while workers are saturated
agent-worker (x3)	LangGraph agents, pulls runs from the queue	Scales with load; CPU and latency heavy
llm-gateway	One interface to a free-tier provider (chosen by env): timeouts, retry with backoff, circuit breaker, token/cost ledger; *stub mode* for load tests	Isolates provider failure; one breaker shared by all workers
redis	Task queue (RQ), embedding/retrieval cache, rate-limit counters	Shared state, different resource profile
chroma · postgres	Vector index · runs, steps, citations, quarantine, feedback, costs	Stateful; independent release
ingest (CLI job)	Parse → clause-aware chunk → embed (all-MiniLM-L6-v2, local) → upsert by content hash	Runs on demand, idempotent
observability	OTel collector → Jaeger, Prometheus, Grafana	Must survive app failures

Runs take tens of seconds, so submission returns a run_id at once and the client **polls** GET /runs/{id}; polling needs no public callback URL in a local setup. Every request carries a correlation ID through api, queue, agents and gateway, so a run can be rebuilt from its run_id (tool calls, queries, passages, prompt version, per-step latency and cost).

4. Agent design (LangGraph)

Intake agent classifies the dispute (tariff category, meter accuracy, load violation, arrears, disconnection) and extracts premises use, sanctioned/connected load and billed category. **Research agent** owns the tools `search_corpus` (hybrid BM25 + dense, with filters), `get_clause(id)`, `get_request_field(name)` and `list_cross_refs(id)`; it retrieves the provision that appears to apply, asks what could modify it (mixed or changed use, load thresholds, exceptions), and searches again with queries built from what it has read. **Verifier agent** checks that every citation ID is in the run's retrieved set and its quoted span appears in that passage, then judges whether the passages support the position. On failure it sends the run back once with a different strategy (clause lookup / keyword search instead of dense); a second failure escalates. A **router** maps the verified position to a route and an output filter blocks any bill revision or amount. Agents hand off through typed state saved to Postgres after each step, so if a worker dies mid-run the run is resumed or marked failed, never returned as an answer. **Loop bounds:** 12 steps, 10 tool calls, 90 s wall clock; hitting any bound escalates with the evidence marked incomplete.

5. Retrieval, evaluation and guardrails

Chunking: split on clause and category headings (about 300–500 tokens) with the parent heading prepended, and keep the notes under each tariff category as their own chunks, since that is where mixed-use rules tend to sit. Metadata: document, section, clause ID, category code. Chunk ID = hash(document version + clause), so only changed chunks are re-embedded. **Prompts** are versioned files; the version is stored with every run. We compare v1 (direct answer) with v2 (explicit “find modifying provisions” checklist). **Evaluation set:** at least 20 scenarios (including 4.1, out-of-corpus connections such as HT and irrigation pump sets, conflicting provisions and injection attempts), each with expected route and supporting passage, run as a CLI batch with TruLens: groundedness, context relevance, faithfulness and hallucination rate (share of claims or citations not backed by retrieved text). Deliberate change: single-pass retrieval vs the agentic re-search, all four metrics before and after. **Guardrails:** injection screening on every free-text field, per-agent tool allow-lists, output filter, toxicity check on input and output.

6. Service level objective (declared before any measurement)

p95 of a complete run (submission to final result) ≤ 60 s at 10 concurrent runs, error rate < 1 %, measured with Locust against the stub LLM (1.5 s per call). Disputes are handled by a grievance officer working through a queue, not by a consumer

waiting live, but the officer reviews the result in the web client with the case open, so about a minute is the useful limit. We will report RPS, p50 and p95, the concurrency at which the target fails and the component responsible, and the effect of scaling agent-worker from 1 to 3.

7. Timeline and prioritisation

Week	Goal
W1 (3–9 Nov)	Corpus download and manifest, locate 4.1 passages, data contract, repo skeleton, compose with api, redis, worker, chroma
W2	**Working end-to-end path:** ingest → one agent → cited answer through the API and a basic web client
W3	Research and Verifier agents, routing and escalation, Postgres run log, guardrails
W4	LLM gateway resilience (timeouts, backoff, breaker, stub), OTel/Jaeger/Prometheus/Grafana, evaluation set and batch CLI
W5	Prompt comparison, before/after change, Locust and scaling, caching and rate limiting, cost per 100 runs
W6	Runbook and incident log, clean-clone README test, report, rehearsal of all 10 demonstration items

Priority 1 (system must start and answer): F1–F7, A1–A4, D1–D4, L1. **Priority 2:** A5, A8, D5, D6, L2–L4, F8. **Priority 3:** A6, A7, D7, D8, L5, L6. The incident log is kept from week 1 so real failures are recorded as they happen.

8. Work split

Member	Owns
Dhanush S	Corpus and ingestion pipeline (D2, L1), agent graph and tools (A1–A3, F3–F7), prompt versions and evaluation (L2, L3), guardrails (L4)
Vidit Soni	API, data contract and quarantine (F1, F2, D1, A4), Compose and LLM gateway (D3, D4, A5), observability and dashboard (D5, A6, F8), Locust, scaling, caching, rate limiting (D6, A7, D7), cost and feedback (L5, L6)
Both	Web client (A8), runbook and incident log (D8), report, demonstration

9. Risks

The mixed-use rule may be vague or split across both documents: we treat that as a valid escalation and confirm the passages at Evaluation 1. Free-tier rate limits: stub for load tests, caching, per-client limits, provider switch by configuration. Scanned pages: OCR fallback. Laptop memory with about ten containers: small embedding model, memory limits in Compose.